Assignment 3: Database Design Analysis

Hospital Management System - redundancy, anomalies, functional dependencies and design improvements

1. Objective and Scope

This report analyses the finalized relational schema of the Hospital Management System (10 relations: PATIENTS, DEPARTMENTS, DOCTOR, APPOINTMENT, ADMISSIONS, MEDICAL_RECORD, BILL, PRESCRIPTION, MEDICINES, PRESCRIPTION_MEDICINE). It identifies redundancy and anomaly risks, lists the functional dependencies (FDs), explains how poor design causes anomalies, and proposes concrete improvements.

2. Functional Dependencies in the Current Design

A functional dependency X -> Y means that the value of X uniquely determines the value of Y. The dependencies below follow from the business rules in the design document (each key attribute identifies one row; each foreign key points to exactly one parent).

Relation	Functional dependencies	Highest normal form
PATIENTS	patient_id -> name, DOB, Gender, ph_no, address	BCNF
DEPARTMENTS	department_id -> department_name, location; department_name -> department_id (names are unique)	BCNF
DOCTOR	doctor_id -> doctor_name, specialization, ph_no, email, department_id; email -> doctor_id; ph_no -> doctor_id	BCNF
APPOINTMENT	appointment_id -> appointment_date, appointment_status, patient_id, doctor_id	BCNF
ADMISSIONS	admission_id -> admission_date, discharge_date, room_number, patient_id, doctor_id	BCNF
MEDICAL_RECORD	record_id -> diagnosis, treatment, record_date, patient_id, doctor_id	BCNF
BILL	bill_id -> total_amount, payment_status, bill_date, patient_id, admission_id; **admission_id -> patient_id** (an admission belongs to one patient)	2NF only: transitive dependency bill_id -> admission_id -> patient_id
PRESCRIPTION	prescription_id -> prescription_date, patient_id, doctor_id	BCNF
MEDICINES	medicine_id -> medicine_name, manufacturer, price, stock_quantity; (medicine_name, manufacturer) -> medicine_id	BCNF
PRESCRIPTION_MEDICINE	(prescription_id, medicine_id) -> quantity (full dependency, no partial dependency)	BCNF

Observation. The design is largely normalized: nine of the ten relations are in BCNF. The only formal violation is in BILL, where patient_id is determined by admission_id (a non-key attribute), giving a transitive dependency. The remaining weaknesses are about *missing links*, *derived data* and *non-atomic data* rather than classic normal-form violations (Section 4).

3. Data Redundancy and Anomalies

3.1 What goes wrong when a design is poor

Redundancy means the same fact is stored in more than one place. Whenever a fact is duplicated, every copy must be kept identical. The DBMS cannot do this on its own, so the application has to - and when it fails, the database contradicts itself. This produces three kinds of anomaly:

- **Insertion anomaly** - a fact cannot be recorded until an unrelated fact exists (e.g. a new doctor cannot be stored until he or she has an appointment), or dummy/NULL values must be invented.
- **Deletion anomaly** - deleting one fact unintentionally destroys another (e.g. deleting the only appointment of a doctor also erases the doctor).
- **Update anomaly** - one real-world change needs many row changes; if any row is missed the data becomes inconsistent (e.g. a department moves to a new block but only some rows are updated).

3.2 Worked example: the same data in a single flat table

To demonstrate the problem, suppose the hospital had stored appointments in one flat table, **APPOINTMENT_FLAT**, instead of the four tables PATIENTS, DOCTOR, DEPARTMENTS and APPOINTMENT. Sample rows (taken from the project data):

appt_id	date	pat_id	patient	doc_id	doctor	specialty	dept	dept_location
5001	2026-09-02	1001	Ravi Shankar	101	Dr. Anil Kumar	Cardiologist	Cardiology	Block A, Floor 2
5013	2026-10-01	1001	Ravi Shankar	101	Dr. Anil Kumar	Cardiologist	Cardiology	Block A, Floor 2
5002	2026-09-03	1003	Mohan Das	101	Dr. Anil Kumar	Cardiologist	Cardiology	Block A, Floor 2
5003	2026-09-05	1004	Sneha Kulkarni	108	Dr. Lakshmi Iyer	General Physician	General Medicine	Block B, Floor 1
5004	2026-09-08	1007	Karthik Naidu	106	Dr. Priya Menon	Pediatrician	Pediatrics	Block C, Floor 1

FDs in the flat table (columns abbreviated): appt_id -> all columns; pat_id -> patient; doc_id -> doctor, specialty, dept; dept -> dept_location. The last chain is *transitive* (appt_id -> doc_id -> dept -> dept_location) and the patient/doctor columns are repeated for every appointment, so the table is only in 1NF.

Anomaly	Scenario in the flat table	Consequence
Redundancy	Ravi Shankar (1001) and Dr. Anil Kumar (101) appear in rows 5001 and 5013; Cardiology's location appears in every cardiology row.	Wasted storage; more places to keep correct.
Update	Cardiology moves from 'Block A, Floor 2' to 'Block D'. Rows 5001, 5013 and 5002 must all be changed.	If one row is missed, the same department has two locations.
Insertion	A new doctor (or the new department 'Oncology') joins, but has no appointments yet.	Cannot be stored without a fake appointment; appt_id (the key) would be NULL, which is illegal.
Deletion	Appointment 5004 (the only one for Dr. Priya Menon, Pediatrics, patient 1007) is cancelled and deleted.	The doctor, the Pediatrics department and the patient's details are all lost as a side effect.

3.3 How the finalized design avoids this

The finalized design decomposes the flat table so that each fact is stored once: patient facts in PATIENTS, doctor facts in DOCTOR, department facts in DEPARTMENTS, and only the keys plus appointment-specific data in APPOINTMENT. Moving Cardiology is now a **single-row** UPDATE of DEPARTMENTS; a doctor or department can be inserted with no appointments; deleting an appointment removes only that appointment. The foreign keys with ON DELETE RESTRICT additionally stop a department or doctor being deleted while child rows still depend on it.

4. Weaknesses Found in the Current Design

The table below lists the redundancy and anomaly risks that remain in the finalized schema and in the supporting ER documents.

#	Issue	Where	Risk / anomaly
1	**Transitive dependency.** BILL stores both admission_id and patient_id, but admission_id already determines the patient.	BILL	Update/insertion anomaly: a bill can reference admission 7001 (patient 1003) while patient_id says 1005. The database accepts the contradiction.
2	**Derived data stored.** total_amount is a calculated value (room, medicines, procedures).	BILL.total_amount	Update anomaly: if a line item changes, the stored total becomes stale. There are no line items to recompute it from.
3	**Documented relationships with no foreign key.** The cardinality table defines ADMISSIONS 1:M MEDICAL_RECORD and MEDICAL_RECORD 1:M PRESCRIPTION, but MEDICAL_RECORD has no admission_id and PRESCRIPTION has no record_id.	MEDICAL_RECORD, PRESCRIPTION	The relationship exists only on paper. Records cannot be tied to a stay; the link must be guessed from patient_id and dates, which is unreliable.

#	Issue	Where	Risk / anomaly
4	**Historical price lost.** price lives only in MEDICINES; PRESCRIPTION_MEDICINE stores only quantity.	MEDICINES, PRESCRIPTION_MEDICINE	Update anomaly: changing a medicine's price silently changes the apparent cost of every past prescription.
5	**room_number is a free-text attribute.** Room facts (ward type, daily tariff, capacity) have no home.	ADMISSIONS	If room attributes are added to ADMISSIONS they repeat for every admission (room_number -> room_type, tariff); a typo ('A-21' vs 'A-201') creates a phantom room.
6	**Non-atomic address** (street, area, city, PIN in one column).	PATIENTS.address	Violates 1NF in spirit; cannot filter or report by city/PIN without string parsing; inconsistent spellings.
7	**Appointment has a date but no time** and no uniqueness rule.	APPOINTMENT	Insertion anomaly: two patients can be booked with the same doctor at the same moment; no slot can be enforced.
8	**Reference ERD (Section 8) contradicts the schema.** It shows appointment_id, admission_id, record_id, prescription_id inside PATIENTS and DOCTOR, department's doctor_id, and FK columns inside MEDICINES.	ER reference diagram	Placing keys of the 'many' side inside the 'one' side creates multi-valued attributes (1NF violation) and update anomalies. The relational schema (Section 5) is correct; the reference ERD should be corrected.
9	**Domain values are not controlled** (status, gender, payment_status).	Several tables	'paid', 'Paid' and 'PAID' can coexist and break grouping/reporting.

5. Proposed Improvements

Each improvement below keeps the design in 3NF/BCNF and removes one of the weaknesses above. SQL shown is standard and runs on MySQL 8 / SQLite.

5.1 Remove the BILL transitive dependency (fixes issue 1)

Patient and admission are both needed (outpatient bills have no admission), so patient_id cannot simply be dropped. Instead, make the database enforce consistency with a composite foreign key:

```
ALTER TABLE ADMISSIONS ADD CONSTRAINT uq_adm_patient UNIQUE (admission_id, patient_id);

-- in BILL (admission_id is nullable; a NULL part skips the check for OPD bills)
FOREIGN KEY (admission_id, patient_id) REFERENCES ADMISSIONS (admission_id, patient_id)
```

Alternative (strict 3NF): split into two tables - BILL(bill_id, patient_id, ...) for patient-level bills and ADMISSION_BILL(bill_id, admission_id) - and derive the patient through the admission.

5.2 Implement the missing relationships (fixes issue 3)

```
ALTER TABLE MEDICAL_RECORD ADD COLUMN admission_id INT NULL,
    ADD CONSTRAINT fk_rec_adm FOREIGN KEY (admission_id) REFERENCES ADMISSIONS(admission_id);
ALTER TABLE PRESCRIPTION  ADD COLUMN record_id INT NULL,
    ADD CONSTRAINT fk_rx_rec  FOREIGN KEY (record_id) REFERENCES MEDICAL_RECORD(record_id);
```

5.3 Replace the stored total with line items (fixes issue 2)

Add a BILL_ITEM table; total_amount becomes SUM(amount) computed by a view or kept only as an audited, trigger-maintained cache.

```
CREATE TABLE BILL_ITEM (
    bill_id      INT            NOT NULL,
    item_no      INT            NOT NULL,
    description  VARCHAR(100)   NOT NULL,
    amount       DECIMAL(10,2)  NOT NULL CHECK (amount >= 0),
    PRIMARY KEY (bill_id, item_no),
    FOREIGN KEY (bill_id) REFERENCES BILL(bill_id) ON DELETE CASCADE
);
```

5.4 Snapshot the price at prescription time (fixes issue 4)

This is deliberate, controlled redundancy: the price at the time of prescribing is a different fact from today's catalogue price.

```
ALTER TABLE PRESCRIPTION_MEDICINE ADD COLUMN unit_price DECIMAL(8,2) NOT NULL CHECK (unit_price > 0);
```

5.5 Introduce a ROOMS table (fixes issue 5)

```
CREATE TABLE ROOMS (
    room_number  VARCHAR(10)   PRIMARY KEY,
    room_type    VARCHAR(20)   NOT NULL CHECK (room_type IN ('General','Semi-Private','Private','ICU')),
    daily_rate   DECIMAL(8,2)  NOT NULL CHECK (daily_rate > 0)
);
-- ADMISSIONS.room_number then becomes a FOREIGN KEY to ROOMS(room_number)
```

5.6 Smaller fixes (issues 6-9)

- **Atomic address:** split PATIENTS.address into street, area, city, pin_code.
- **Time slots:** add appointment_time and UNIQUE (doctor_id, appointment_date, appointment_time) so a doctor cannot be double-booked.
- **Controlled domains:** keep the CHECK constraints on status/gender columns (already in the DDL) or move them to small lookup tables if values must change without schema changes.
- **Indexes:** create indexes on foreign-key columns used for joins (e.g. APPOINTMENT.patient_id, APPOINTMENT.doctor_id, BILL.patient_id).
- **Documentation:** correct the reference ERD so foreign keys appear only on the 'many' side, matching Section 5 of the design document.

6. Before and After Summary

Aspect	Current design	Improved design
BILL patient consistency	Not enforced (transitive dependency)	Composite FK guarantees bill and admission refer to the same patient
Billing total	Stored, can go stale	Derived from BILL_ITEM
Record - admission - prescription links	Documented only	Enforced by foreign keys
Past prescription cost	Changes when price changes	Frozen in unit_price
Room data	Free text	Validated through ROOMS
Double booking	Possible	Prevented by unique slot constraint
Normal form	9 of 10 tables BCNF, BILL in 2NF	All tables in 3NF/BCNF

7. Conclusion

Redundancy is the root cause of insertion, deletion and update anomalies: duplicated facts must be changed in many places, and facts bundled in one table cannot be added or removed independently. The Hospital Management System is already well normalized because each entity has its own table and relationships are expressed through foreign keys. The remaining improvements - enforcing the BILL/ADMISSIONS dependency, adding the missing foreign keys, replacing stored totals with line items, snapshotting prices, and adding a ROOMS table - move the design from "mostly normalized" to a design where the DBMS itself guarantees consistency.